

Tarea 2 2026

Introducción a la Computación Paralela

Procesamiento de imágenes en CUDA

Cecilia Hernández y Álvaro Guzmán

Junio 2026

1 Descripción General

El objetivo de esta tarea es implementar, analizar y optimizar el procesamiento masivo de un gran volumen de imágenes a color utilizando la arquitectura de GPU mediante CUDA. Específicamente, se busca extraer información estadística del conjunto de datos calculando la matriz de covarianza de las imágenes centradas.

Para este propósito, cada imagen tridimensional se interpretará como un vector aplanado unidimensional. Si se tiene un conjunto de m imágenes, y cada imagen aplanada tiene tamaño n (ancho $\times$ alto $\times$ canales), denotamos el k -ésimo vector de imagen como $v^{(k)}$.

El cálculo requiere los siguientes pasos matemáticos:

1. **Vector Promedio:** Obtener el valor promedio a través de todo el conjunto de imágenes para cada componente j :

$$\mu_j = \frac{1}{m} \sum_{k=0}^{m-1} v_j^{(k)}$$

2. **Matriz Centrada:** Restar el vector promedio a cada imagen original para obtener las imágenes centradas:

$$\bar{v}_j^{(k)} = v_j^{(k)} - \mu_j$$

3. **Matriz de Covarianza:** Finalmente, calcular la matriz de covarianza C de tamaño $n \times n$, donde cada elemento está dado por la suma del producto punto sobre todas las m muestras:

$$C_{jj'} = \frac{1}{m} \sum_{k=0}^{m-1} \bar{v}_j^{(k)} \cdot \bar{v}_{j'}^{(k)}$$

Dado el gran volumen de datos, se provee el siguiente esquema base utilizando la biblioteca `CImg.h` para la lectura de imágenes y la preparación de los punteros, el cual debe expandirse:

```
1 #define cimg_display 0
2 #define cimg_use_jpeg
3 #include "CImg.h"
4 #include <iostream>
5 #include <vector>
6
7 using namespace cimg_library;
8
9 int main() {
10     int num_images = 10000; // Ejemplo de volumen masivo
11     int width, height, channels;
12
13     // Asignar memoria paginada bloqueada (Pinned Memory) para Streams
```

```

14 float *h_dataset;
15 // cudaMallocHost((void*)&h_dataset, num_images * n * sizeof(float));
16
17 // Bucle para cargar y aplanar las imagenes
18 for (int k = 0; k < num_images; k++) {
19     std::string filename = "dataset/img_" + std::to_string(k) + ".png";
20     CImg<unsigned char> img(filename.c_str());
21
22     if (k == 0) {
23         width = img.width();
24         height = img.height();
25         channels = img.spectrum();
26     }
27
28     // AQUI VA SU CODIGO:
29     // 1. Convertir 'img' a escala de grises si es necesario.
30     // 2. Poblar el arreglo lineal h_dataset con los valores de la imagen.
31 }
32
33 // AQUI VA SU CODIGO CUDA:
34 // (Cálculo del promedio, centrado y Matriz de Covarianza)
35
36 // cudaFreeHost(h_dataset);
37 return 0;
38 }

```

2 Objetivos

- Implementar correctamente operaciones de reducción matemática y álgebra lineal densa en la GPU utilizando CUDA.
- Comprender las limitaciones físicas de la memoria VRAM y el bus PCIe frente a grandes conjuntos de datos.
- Aplicar paralelismo asíncrono y ocultación de latencia mediante particionamiento de datos y el uso de múltiples CUDA Streams.
- Realizar un análisis crítico del rendimiento, evaluando el impacto temporal del solapamiento de copias y ejecución de kernels. Puede usar un profiler para CUDA de Nsight system (nsys) and Nsight compute (ncu).

3 Primer Experimento: Implementación Tradicional en CUDA

[2.0 puntos] Diseñe e implemente un código en CUDA que resuelva el problema asumiendo que todo el conjunto de datos puede cargarse de forma contigua en la GPU (utilizando el Stream 0 por defecto).

Su programa debe alojar toda la memoria requerida en el *device* (`cudaMalloc`), realizar una copia sincrónica del dataset aplanado (`cudaMemcpy`), lanzar los kernels correspondientes para calcular el vector promedio, centrar los datos y finalmente realizar la multiplicación matricial para la covarianza mediante memoria compartida (Tiling). Mida y reporte los tiempos de copia hacia la GPU, tiempo neto de cómputo (ejecución de kernels) y tiempo de copia de la matriz C resultante de vuelta al host.

4 Segundo Experimento: Orquestación con CUDA Streams

[3.0 puntos] Debido a que la adición es una operación asociativa, la sumatoria para el cálculo de la covarianza puede calcularse de forma incremental, permitiendo determinar C procesando pequeñas fracciones de los datos a la vez.

Diseñe una segunda implementación optimizada que divida el volumen masivo de imágenes en *batches* (lotes) y utilice *CUDA Streams* concurrentes.

- Asegúrese de que la memoria en el host sea paginada bloqueada (*Pinned Memory*) utilizando `cudaMallocHost`.
- Utilice transferencias asíncronas (`cudaMemcpyAsync`) para solapar la copia del *batch* $s+1$ mientras el *batch* s está siendo procesado matemáticamente en la GPU.
- Permita que la cantidad de streams S sea un parámetro configurable en tiempo de ejecución.

5 Datos

Para el desarrollo de esta tarea debe utilizar el conjunto de datos DIV2K (DIVERse 2K Resolution Image Dataset), disponible públicamente en:

<https://data.vision.ee.ethz.ch/cvl/DIV2K/>

DIV2K es un conjunto de imágenes naturales de alta calidad ampliamente utilizado en aplicaciones de procesamiento de imágenes y visión computacional. El dataset se distribuye en diferentes particiones de entrenamiento y validación, así como en múltiples resoluciones generadas mediante distintos factores de escalamiento.

Con el propósito de facilitar la experimentación y mantener tiempos de ejecución razonables, se recomienda utilizar la partición de validación de baja resolución obtenida mediante reducción bicúbica con factor de escalamiento 4 (DIV2K_valid_LR_bicubic_X4). Esta partición contiene 100 imágenes a color en formato PNG, proporcionando un volumen de datos suficientemente grande para evaluar estrategias de procesamiento paralelo y transferencia de datos entre CPU y GPU.

Podrán utilizar la totalidad del conjunto de validación o una fracción representativa del mismo, siempre que se justifique adecuadamente la configuración experimental adoptada. Asimismo, podrán aplicar técnicas de preprocesamiento tales como redimensionamiento, extracción de subimágenes o particionamiento en bloques, cuando estas sean necesarias para adaptar el problema a las restricciones de memoria de la GPU utilizada.

6 Informe

[1.0 puntos] Redacte un informe detallando sus decisiones de implementación algorítmica y orquestación de memoria. Debe indicar explícitamente el entorno de ejecución (CPU, modelo de GPU, cantidad de RAM y VRAM disponible).

Ejecute experimentos variando la cantidad de streams concurrentes (por ejemplo, $S \in \{1, 2, 4, 8, 16\}$, considerando $S = 1$ como su línea base o ejecución sin solapamiento) y reporte:

- Tablas con los tiempos de ejecución totales para cada configuración.
- Un gráfico que ilustre el *Speedup* obtenido en función de la cantidad de streams.
- Un diagrama o esquema (basado en herramientas de *profiling* como `nvprof` o *Nsight Systems*) que evidencie los tiempos de ocupación del host y la GPU, demostrando el nivel de solapamiento logrado.

7 Conclusión y Análisis

El informe debe cerrar con una discusión crítica de los resultados empíricos frente a los teóricos. ¿A partir de cuántos streams el rendimiento dejó de mejorar y por qué? Discuta cómo la limitación del bus PCIe fue mitigada (o no) a través de la estrategia de *latency hiding* y concluya sobre la viabilidad de utilizar esta arquitectura para procesar conjuntos de datos que superan la memoria física de la tarjeta gráfica.

8 Criterios de evaluación

Se evaluará:

- corrección de las implementaciones;
- calidad del análisis teórico;
- calidad y claridad de la evaluación experimental;
- profundidad de la discusión de resultados;
- presentación general del informe.